Session 1 Agenda

1. From servlets to a framework — the bridge
2. MVC the **pattern** vs Spring MVC the **framework**
3. The Front Controller pattern
4. `DispatcherServlet` — Spring's front controller
5. **Three containers**: `ServletContext`, root context, web context
6. `ContextLoaderListener` and the two default file names
7. **Can I skip the root context?** + the startup order
8. **Three ways to configure** `DispatcherServlet` + URL patterns
9. Running it on Tomcat
10. **The full request lifecycle**

You Already Know Servlets

One servlet handles one URL. You wrote `doGet` / `doPost`, read parameters from `HttpServletRequest`, wrote output to `HttpServletResponse`, and mapped it in `web.xml`.

```
@WebServlet("/accounts")
public class AccountServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) {
        String id = req.getParameter("id");
        // ... find the account
        req.setAttribute("account", account);
        req.getRequestDispatcher("/WEB-INF/views/account.jsp").forward(req,
resp);
    }
}
```

This works. So what's the problem?

The Problem: 50 URLs → 50 Servlets

Every servlet repeats the same chores:

- Parse parameters by hand, convert `String` → `int` / `Date` / your object

- Decide which JSP to forward to, by hand
- Validate input, by hand
- Handle exceptions, by hand
- Map each one separately in `web.xml`

The **business logic** in each servlet is maybe 3 lines. Everything else is **plumbing repeated 50 times.**

Sound familiar? Same shape of problem as the one AOP solved — repeated mechanical code that isn't what the class is really *about*.

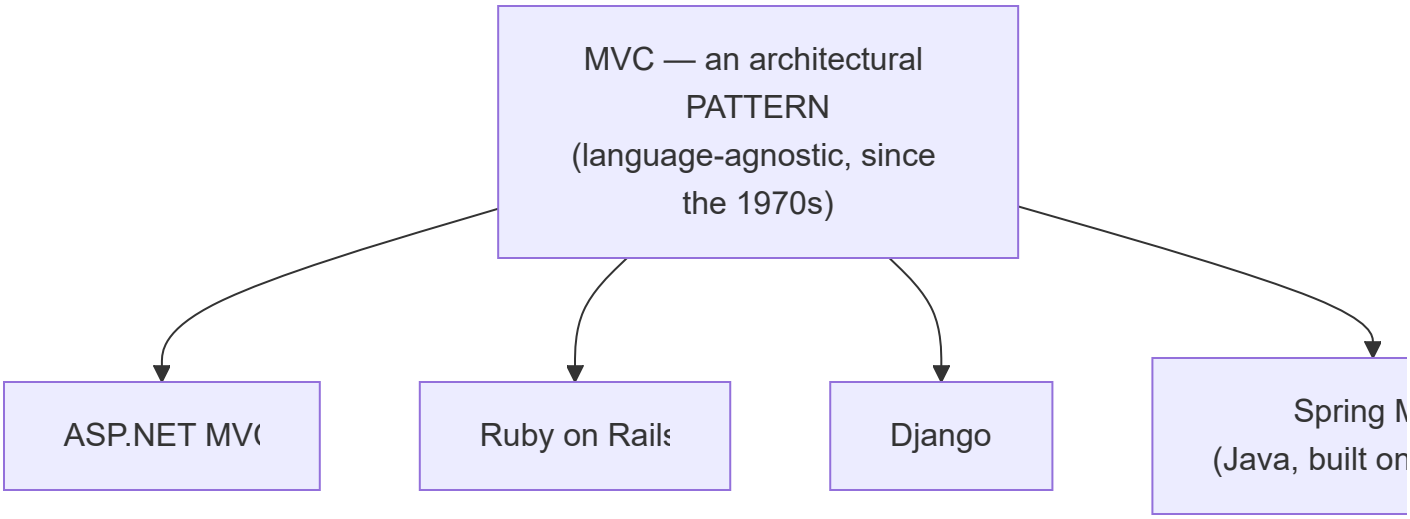
MVC — The Pattern (Not The Framework)

MVC is an architectural pattern, invented in the 1970s (Smalltalk), used in hundreds of frameworks across many languages — ASP.NET MVC, Ruby on Rails, Django, Angular, Laravel…

Part	Responsibility
Model	the data + business state (your `Account`, `Order`, service results)
View	how that data is *presented* (JSP page, HTML, JSON)
Controller	receives input, coordinates: asks the model, picks the view

The whole point: **separation of concerns.** The view doesn't know how data was fetched. The model doesn't know how it will be displayed.

So Where Does "Spring MVC" Fit?



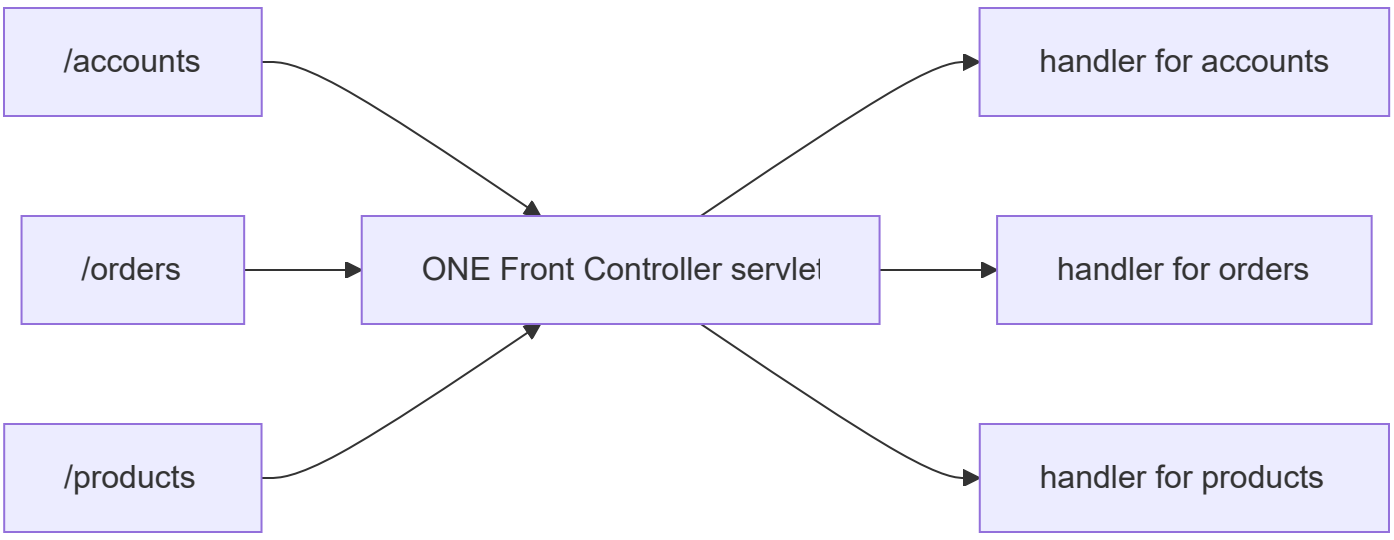
Spring MVC = Spring's implementation of the MVC pattern for Java web apps.

- It does not replace servlets — it is **built on top of** the Servlet API
- It implements MVC using the **Front Controller** pattern
- That front controller is a class called `DispatcherServlet` — and yes, it is a real `HttpServlet`, just like the ones you already wrote

---

The Front Controller Pattern

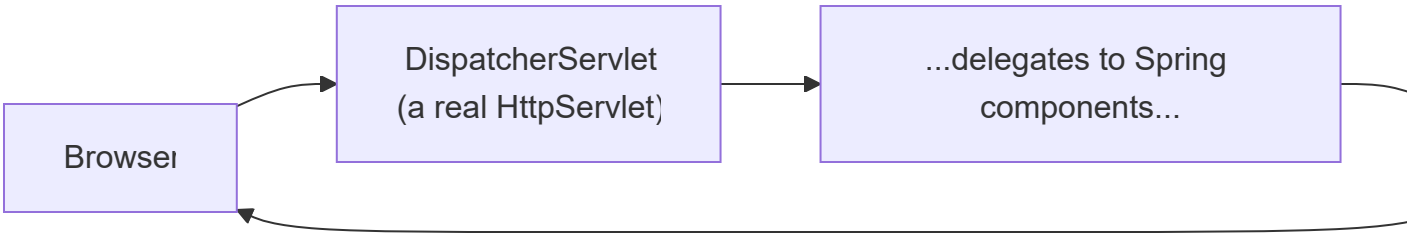
Instead of many servlets each handling one URL:



One servlet receives every request, then routes it to the right handler.

Because all traffic flows through one place, that one place can do the shared plumbing **once**: parameter binding, validation, exception handling, view resolution — for everybody.

`DispatcherServlet` — Spring's Front Controller



- It is mapped to a broad URL pattern (usually `/`)
- It **does not contain your business logic** — it *orchestrates*
- It delegates every step to a pluggable component: finding the handler, invoking it, resolving the view
- Every one of those components is an **interface with multiple implementations** — which is what makes Spring MVC so configurable

---

Two Kinds of Spring Container in a Web App

A Spring web application has **two kinds of container**, each configured and initialised differently:

- the **Application Context** — usually called the **root context**
- the **Web Application Context** — one per `DispatcherServlet`

One trap before we go further: `WebApplicationContext` **is also the name of a Java interface**, and *both* of these are instances of it.

```
public interface WebApplicationContext extends ApplicationContext {
    ServletContext getServletContext();
}
```

So "Application Context" vs "Web Application Context" describes a **role**, not a type.

First — What Is the `ServletContext`?

`ServletContext` **is a Servlet API object, not a Spring one.** It exists whether or not Spring is on the classpath.

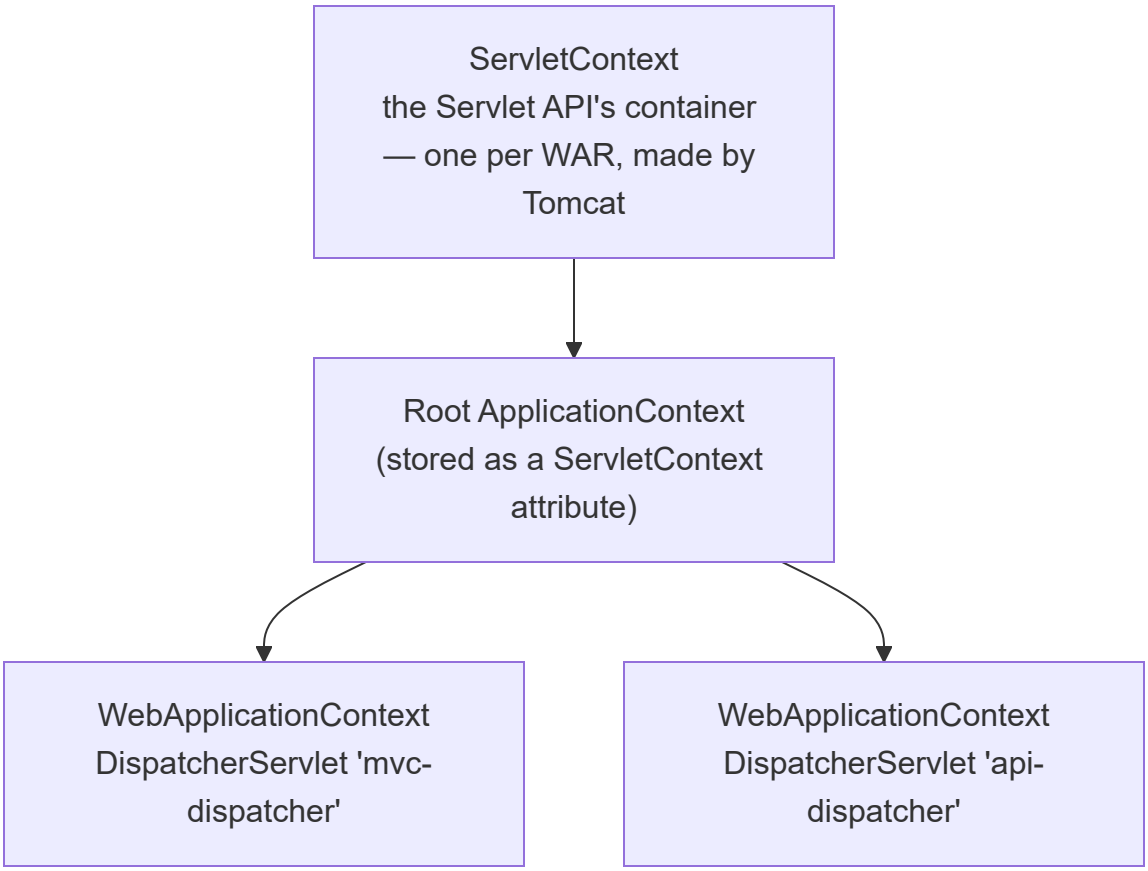
- **One per deployed web application** (one per WAR), created by Tomcat
- It is the application's shared map: `getAttribute` / `setAttribute`, init-params, real paths, logging
- It knows nothing about beans, injection, or Spring

Spring does not replace it. Spring **parks its containers inside it**:

`ServletContext` attribute key	Holds
`org.springframework.web.context.WebApplicationContext.ROOT`	the root context
`org.springframework.web.servlet.FrameworkServlet.CONTEXT.<servlet-name>`	that servlet's web context

That is how `WebApplicationContextUtils.getWebApplicationContext(servletContext)` can hand the root context to a plain filter, a plain servlet, or a JSP — code that has never heard of Spring's DI.

Three Containers, Stacked



- `ServletContext` **holds** the Spring contexts — it is not their parent in the Spring sense

- The root context **is** the Spring parent of every web context
- **One root context per application. One web context per `DispatcherServlet`.**

The Root Context — Declared in `web.xml`

```
<listener>
    <listener-class>
        org.springframework.web.context.ContextLoaderListener
    </listener-class>
</listener>

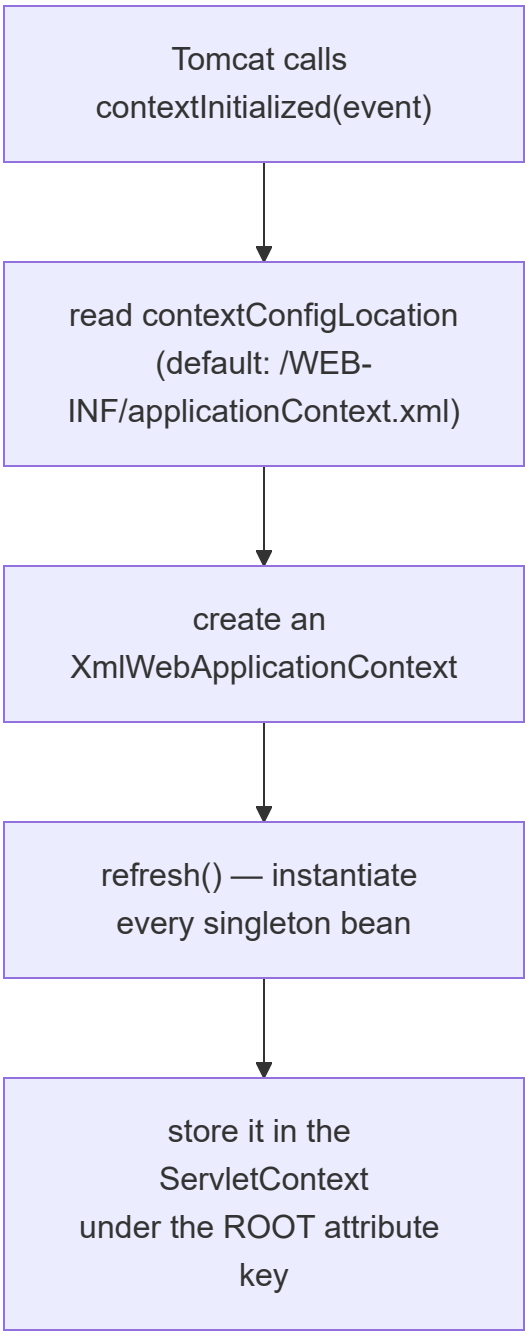
<!-- optional — only needed to override the default location -->
<context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/root-context.xml</param-value>
</context-param>
```

What belongs here: middle-tier transactional services, data access objects, the `DataSource`, the transaction manager, your AOP aspects — anything you want to **use and re-use across the whole application**, not just the web layer.

> **Historical note:** older material offers `ContextLoaderServlet` as an alternative to the listener. It was deprecated in Spring 2.5 and **removed in Spring 3.0**. If you see it, the codebase is very old. Use the listener.

What `ContextLoaderListener` Actually Does

It implements `ServletContextListener`, so Tomcat calls it **once, before any servlet is initialized**.



On shutdown `contextDestroyed` closes the context, so `@PreDestroy` and `destroy-method` actually run.

Because it is a **listener**, not a servlet, it is guaranteed to finish before any `DispatcherServlet` starts and asks for its parent. That ordering is the entire reason it exists.

The Two Default File Names

Neither `contextConfigLocation` is required. Each side has a convention:

Container	Default location	Where the default comes from
Root	`/WEB-INF/applicationContext.xml`	`ContextLoader.DEFAULT_CONFIG_LOCATION`
Web	`/WEB-INF/<servlet-name>-servlet.xml`	`FrameworkServlet` : namespace = servlet-name + `-servlet`

So this declaration:

```
<servlet>
    <servlet-name>mvc-dispatcher</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    <load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
    <servlet-name>mvc-dispatcher</servlet-name>
    <url-pattern>*.htm</url-pattern>
</servlet-mapping>
```

…looks for `/WEB-INF/mvc-dispatcher-servlet.xml`. Rename the servlet to `dispatcher` and it looks for `dispatcher-servlet.xml` instead. The file name follows the **servlet name**, never the class name.

⚠️ Declaring the listener but forgetting the file is a **startup failure** (`FileNotFoundException`), not a silent skip.

`load-on-startup` — Small Tag, Big Effect

```
<load-on-startup>1</load-on-startup>
```

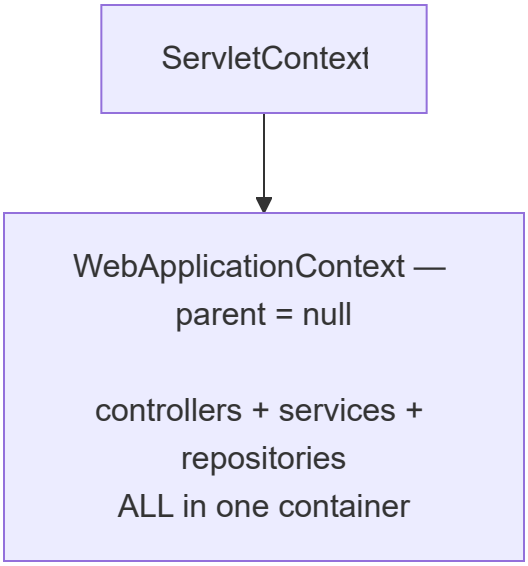
Value	Behaviour
absent, or negative	the servlet is created **on its first request**
0 or positive	created **at deploy time**; lower numbers are created first

For `DispatcherServlet` this decides *when* its web context is built — and therefore **when your bean errors surface**: at deployment where you'll see them, or in front of your first user. Always set it.

Can I Skip the Root Context Entirely?

Yes. And the answer to *"then where do the beans go?"* is: **all of them go into the `DispatcherServlet`'s web context.**

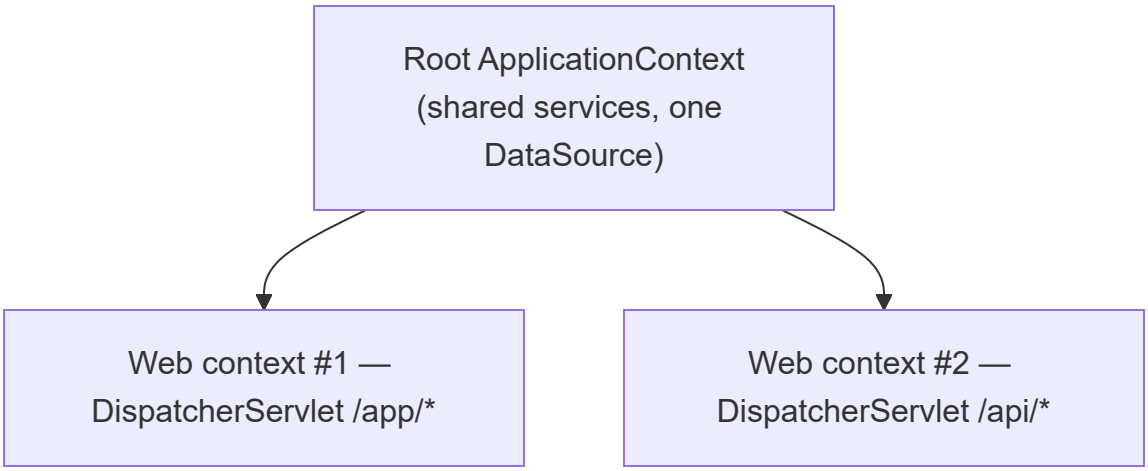
- With no `ContextLoaderListener` registered, **no root context is created**. There is no hidden default and no automatic fallback — `WebApplicationContextUtils.getWebApplicationContext(sc)` simply returns `null`.
- `DispatcherServlet` then builds its own context with `parent == null`, and services, repositories, controllers and view resolvers all live in that one container.
- For a single-`DispatcherServlet` app this is completely normal. **Spring Boot works exactly this way** — one context, no parent/child split at all.



…So When Do I Actually Want Two?

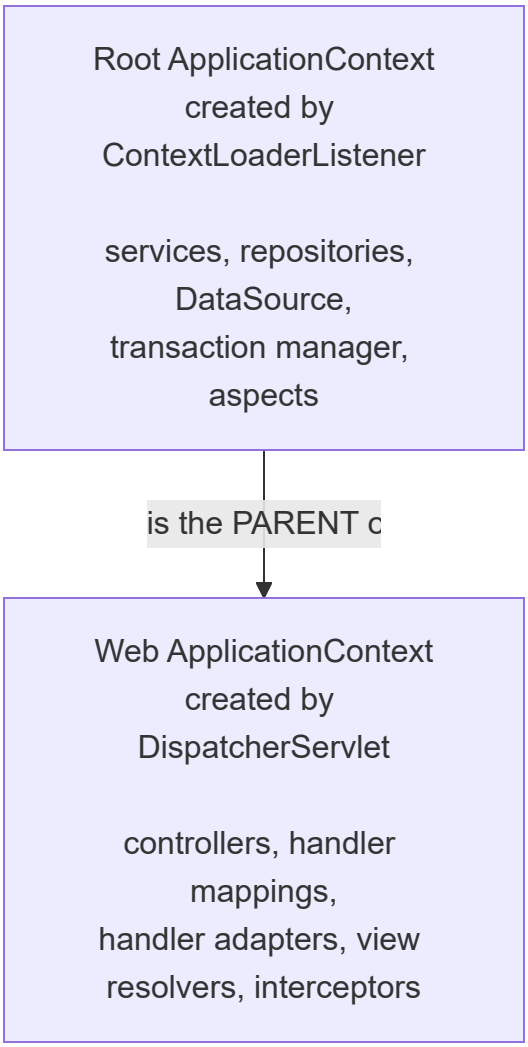
Keep the split when one of these is true:

- **More than one `DispatcherServlet`** — say `/app/*` for the UI and `/api/*` for REST. Each gets its **own** web context; both share **one** root context, so one `DataSource`, one connection pool, one set of services.
- **Non-web entry points** — a scheduled job, a JMS listener, a CLI tool loading the same service layer.
- **A structural guarantee** that the service layer cannot reach into the web layer.



Two things to check if you drop the root context: `DelegatingFilterProxy` (how Spring Security is wired in) looks in the **root** context by default, and so does `RequestContextListener`. With no root context you must point them at a servlet's context via the `contextAttribute` init-param.

The Parent/Child Rule

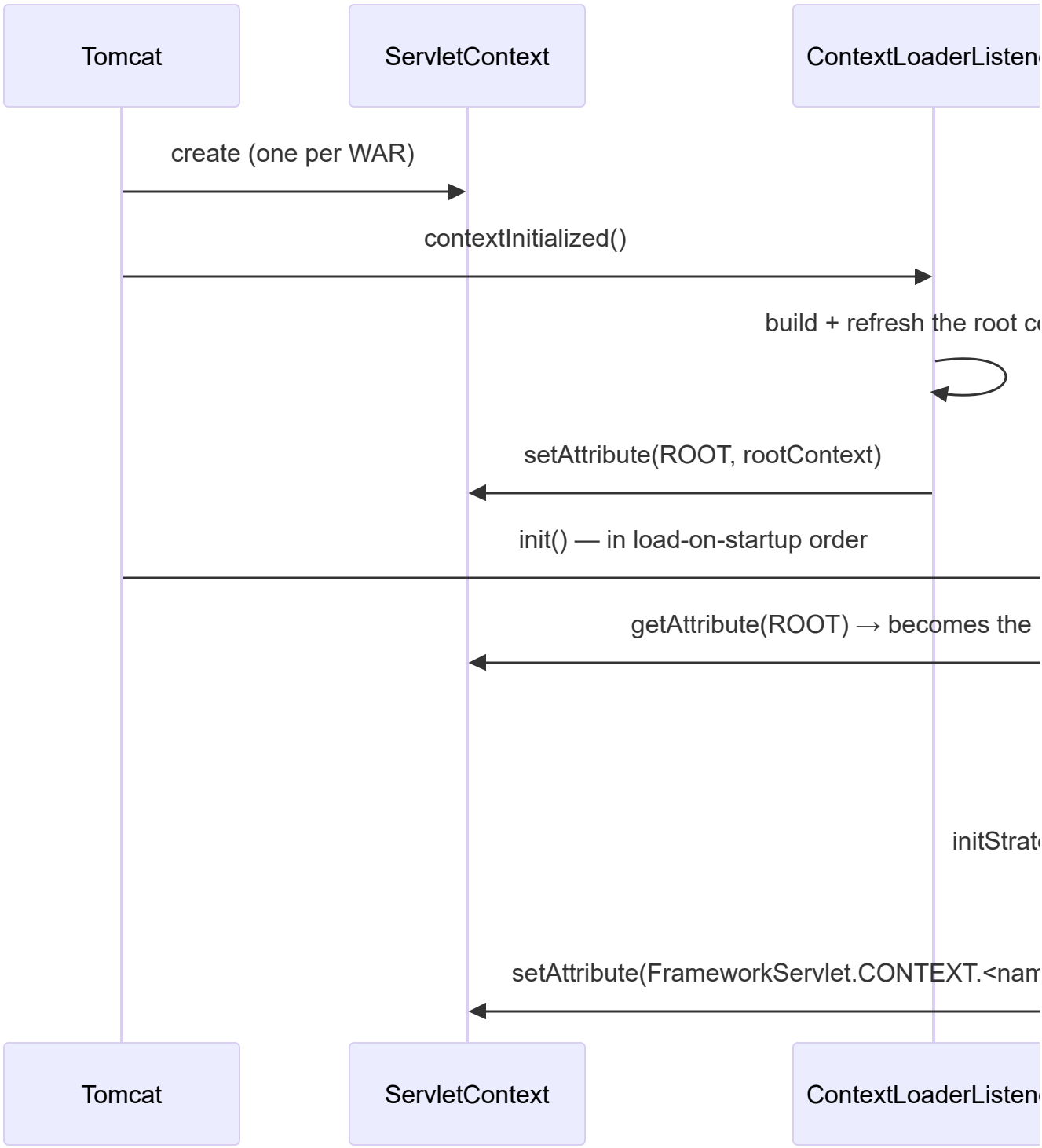


A child can see the parent's beans. A parent cannot see the child's.

- Your controller (web context) **can** inject `AccountService` (root context) ✅
- Your `AccountService` (root context) **cannot** inject a controller ❌

Mechanically: `getBean` looks in the child first and delegates **upward** to the parent on a miss. There is no downward lookup, so the rule falls straight out of the algorithm.

Startup Order, End to End



Everything above happens **once, at deploy time**. None of it is per-request.

Configuring `DispatcherServlet` — Three Ways

Way	Needs	You write
1. `web.xml`	any servlet version	`<servlet>` + `<serv` `mapping>`
2. Implement `WebApplicationInitializer`	Servlet 3.0+	`onStartup(Servlet(` — full manual contro
3. Extend `AbstractAnnotationConfigDispatcherServletInitializer`	Servlet 3.0+	three short methods

All three do the **same thing**: register a servlet with the container and tell it where its configuration lives. Ways 2 and 3 just do it in Java instead of XML — there is no behavioural difference at runtime.

Way 1 — `web.xml`

```
<servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>
        org.springframework.web.servlet.DispatcherServlet
    </servlet-class>
    <init-param>
        <param-name>contextConfigLocation</param-name>
        <param-value>/WEB-INF/dispatcher-servlet.xml</param-value>
    </init-param>
    <load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
    <servlet-name>dispatcher</servlet-name>
    <url-pattern>/</url-pattern>
</servlet-mapping>
```

Way 2 — Implement `WebApplicationInitializer`

```
public class MyInitializer implements WebApplicationInitializer {

    @Override
    public void onStartup(ServletContext servletContext) {
```

```
        AnnotationConfigWebApplicationContext web =
                new AnnotationConfigWebApplicationContext();
        web.register(WebConfig.class);

        ServletRegistration.Dynamic reg =
                servletContext.addServlet("dispatcher", new
DispatcherServlet(web));
        reg.setLoadOnStartup(1);
        reg.addMapping("/");
    }
}
```

This is `web.xml` rewritten in Java, line for line. Reach for it when you need something the base classes don't expose — a conditional registration, several servlets built in a loop, extra listeners.

Way 3 — Extend the Base Class

```
public class MyWebAppInitializer
        extends AbstractAnnotationConfigDispatcherServletInitializer {

    @Override
    protected Class<?>[] getRootConfigClasses() {
        return new Class[]{ RootConfig.class };      // services, repositories
    }

    @Override
    protected Class<?>[] getServletConfigClasses() {
        return new Class[]{ WebConfig.class };       // controllers, view
resolvers
    }

    @Override
    protected String[] getServletMappings() {
        return new String[]{ "/" };                  // DispatcherServlet URL
pattern
    }
}
```

The three methods map exactly onto the three things `web.xml` did: root config, servlet config, servlet mapping.

Return `null` from `getRootConfigClasses()` and you get the single-context setup from a few slides ago — no `ContextLoaderListener` is registered at all.

The URL Pattern Matters More Than It Looks

Pattern	Meaning	JSPs still render?
`/`	replaces the container's **default** servlet	✅ `*.jsp` is a more specific mapping, so Jasper still wins
`/*`	**catch-all** — beats every other mapping	❌ the forward to `.jsp` comes straight back to `DispatcherServlet`
`*.htm`	extension mapping, the classic legacy style	✅ but every URL needs the suffix
`/api/*`	path mapping — ideal for a second dispatcher	✅

Rule of thumb: use `/`, never `/*`.

Running It: Tomcat

Option A — standalone Tomcat (the production picture)

```
mvn package          → target/myapp.war
copy myapp.war → TOMCAT_HOME/webapps/
start Tomcat         → http://localhost:8080/myapp/
```

Option B — Maven plugin (what we'll use in labs)

```
<plugin>
    <groupId>org.codehaus.cargo</groupId>
    <artifactId>cargo-maven3-plugin</artifactId>
    <configuration>
        <container>
            <containerId>tomcat10x</containerId>
            <type>embedded</type>
        </container>
    </configuration>
</plugin>
```

```
mvn clean package cargo:run    → http://localhost:8080/myapp/
```

The Dependencies

```
<packaging>war</packaging>

<dependencies>
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-webmvc</artifactId>   <!-- pulls in core, context,
web, aop -->
        <version>6.1.0</version>
    </dependency>

    <!-- provided: Tomcat already has it at runtime -->
    <dependency>
        <groupId>jakarta.servlet</groupId>
        <artifactId>jakarta.servlet-api</artifactId>
        <version>6.0.0</version>
        <scope>provided</scope>
    </dependency>

    <!-- JSTL for JSP views -->
    <dependency>
        <groupId>jakarta.servlet.jsp.jstl</groupId>
        <artifactId>jakarta.servlet.jsp.jstl-api</artifactId>
        <version>3.0.0</version>
    </dependency>
</dependencies>
```

The Request Lifecycle

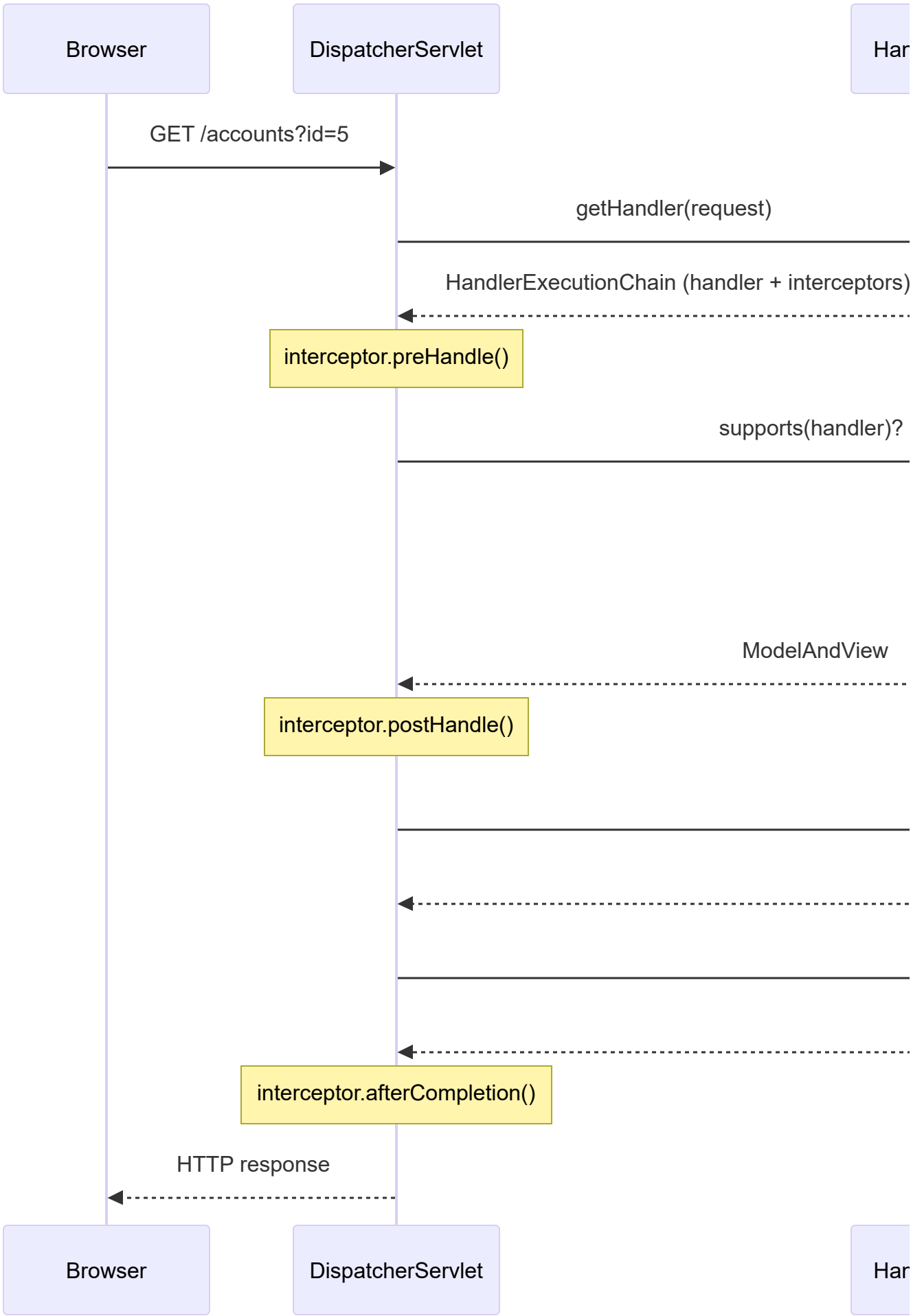
The most important diagram in this module

The Full Lifecycle

“Pasted image 20260918171141.png” could not be found.

Every arrow leaving the dark column is `DispatcherServlet` calling an **interface** it was handed at startup.

The Same Flow, Compressed


Browser
DispatcherServlet
Har
GET /accounts?id=5
getHandler(request)
HandlerExecutionChain (handler + interceptors)
interceptor.preHandle()
supports(handler)?
ModelAndView
interceptor.postHandle()
interceptor.afterCompletion()
HTTP response
Browser
DispatcherServlet
Har

The Same Thing, In Words

1. Request arrives at `DispatcherServlet` (it's mapped to `/`)
2. `HandlerMapping`: "who handles `/accounts`?" → returns a `HandlerExecutionChain`
3. `HandlerAdapter`: finds an adapter that knows how to *invoke* this particular kind of handler
4. The adapter does the heavy lifting: binds request params to method arguments, runs validation, then **calls your controller method**
5. Your controller returns a view name + model (or a `ModelAndView`)
6. If it returned no view name, `RequestToViewNameTranslator` invents one from the URL
7. `ViewResolver`: turns the logical name `"account"` into a real `View` object (e.g. `/WEB-INF/views/account.jsp`)
8. The `View` renders, reading values from the model
9. Response goes back to the browser

Why Is It Built Out of So Many Interfaces?

This is the right question, and the short answer is the one you'd expect: **loose coupling, applied deliberately, at three separate seams.**

Seam	What `DispatcherServlet` is protected from
`HandlerMapping`	*how* a URL is matched — annotations, bean names, an explicit map, a routing function
`HandlerAdapter`	*what shape* a handler is — a method, an interface, a raw servlet-style object
`ViewResolver` / `View`	*what technology* renders the page — JSP, Thymeleaf, PDF, JSON

Look at what `HandlerMapping.getHandler()` actually gives back: the handler inside the chain is typed as `Object`. `DispatcherServlet` is written so that it is *literally incapable* of knowing what your controller is. That is not sloppiness — it is the design.

Why Does Step 4 Even Exist?

> "If `HandlerMapping` already found the handler, why doesn't `DispatcherServlet` just **call** it? Why a separate adapter?"

Because **handlers don't all look the same.** A modern `@Controller` method can have any name and any parameters:

```
@GetMapping("/accounts")
public String list(@RequestParam int id, Model model) { ... }
```

But an old-style handler implements a fixed interface:

```
public class LegacyController implements Controller {
    public ModelAndView handleRequest(HttpServletRequest req,
HttpServletResponse res) { ... }
}
```

`DispatcherServlet` cannot call both of these the same way. It needs a translator. That translator is the **Adapter pattern.**

Why `/WEB-INF/views/`?

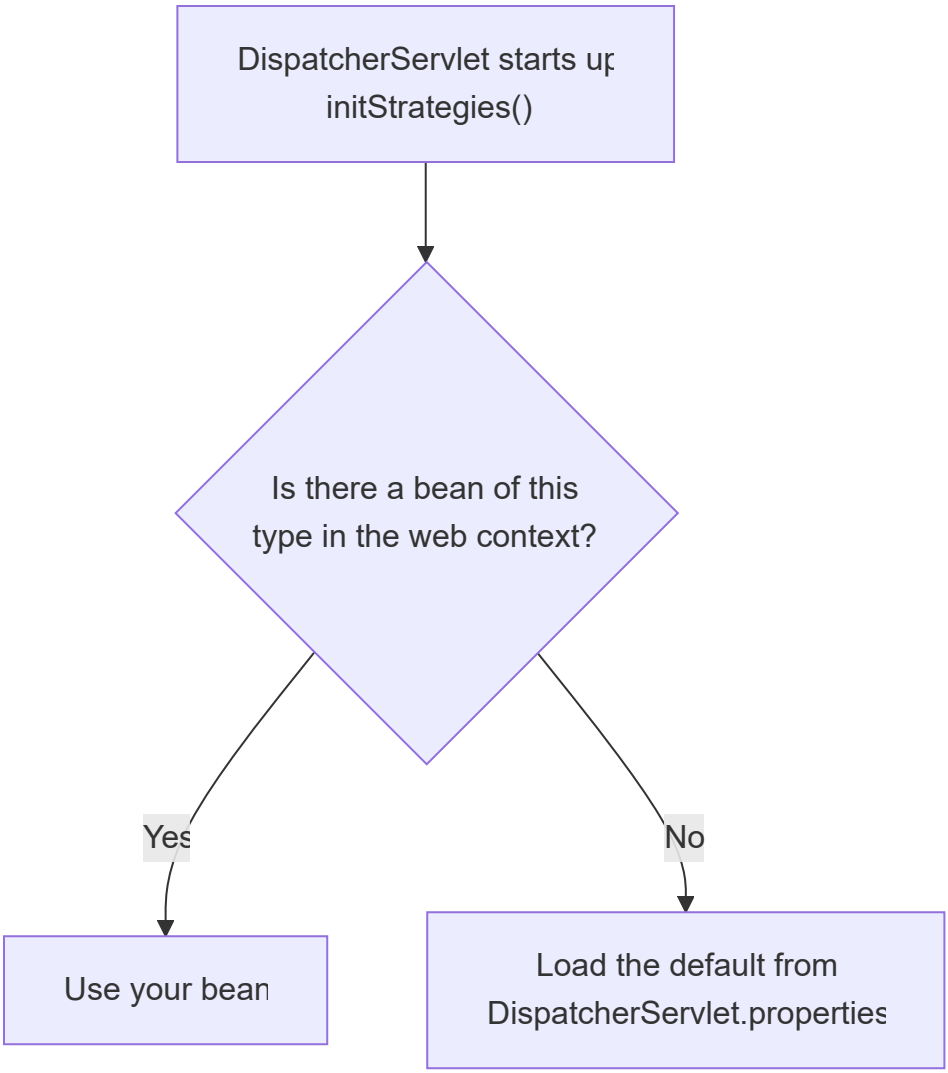
Anything under `/WEB-INF/` **is not directly reachable by a browser** — the servlet container blocks direct URL access to it.

- ✅ User types `/accounts` → controller runs → forwards internally to the JSP
- ❌ User types `/WEB-INF/views/account.jsp` → **404**, blocked by the container

That's exactly what we want: the JSP should only ever be rendered *after* a controller has populated the model. Otherwise you'd get a page full of blanks.

What If I Configure Nothing?

`DispatcherServlet` has a **fallback**. On startup, for each strategy interface it needs, it does this:



`DispatcherServlet.properties` contains the list of default strategy objects used by the `DispatcherServlet`, located in the same package as `DispatcherServlet` itself, and during `initStrategies()` each strategy is checked one by one — if it hasn't already been registered, the strategy from `DispatcherServlet.properties` is used instead.

The file itself carries a comment saying it's used as fallback when no matching beans are found in the DispatcherServlet context, and is not meant to be customized by application developers.

The Actual Defaults

Strategy	Default implementation(s)
`HandlerMapping`	`BeanNameUrlHandlerMapping` and `RequestMappingHandlerMapping`
`HandlerAdapter`	`HttpRequestHandlerAdapter`, `SimpleControllerHandlerAdapter`, and

Strategy	Default implementation(s)
	`RequestMappingHandlerAdapter`
`HandlerExceptionResolver`	`ExceptionHandlerExceptionResolver`, `ResponseStatusExceptionResolver`, `DefaultHandlerExceptionResolver`
`LocaleResolver`	`AcceptHeaderLocaleResolver`
`ViewNameTranslator`	`DefaultRequestToViewNameTranslator`
`ViewResolver`	`InternalResourceViewResolver`

Careful — It's All-or-Nothing, Per Strategy

`initStrategies()` asks *"are there **any** beans of this type?"*, not *"is there one that handles this URL?"*

```
// DispatcherServlet.initHandlerMappings(...)
Map<String, HandlerMapping> matchingBeans = BeanFactoryUtils
        .beansOfTypeIncludingAncestors(context, HandlerMapping.class, true,
false);

if (!matchingBeans.isEmpty()) {
    // use YOURS, sorted by order — the defaults are never loaded
} else {
    // fall back to DispatcherServlet.properties
}
```

So the moment you declare **one** `SimpleUrlHandlerMapping`, you lose `RequestMappingHandlerMapping` **and** `BeanNameUrlHandlerMapping` — meaning your `@Controller` annotations stop being seen entirely.

The same rule applies to `HandlerAdapter`, `ViewResolver`, `HandlerExceptionResolver` and the rest. The demo project triggers this on purpose so you can watch it happen.

So Why Do We Write Any Config At All?

Because the **defaults are bare minimums**:

- The default `ViewResolver` has **no prefix/suffix** — it would treat `"account"` as the literal path `account`, not `/WEB-INF/views/account.jsp`

- Defaults give you no message converters for JSON, no validation setup, no interceptors, no static resource handling

`@EnableWebMvc` **is the upgrade**: it replaces the bare defaults with a properly configured set — a `RequestMappingHandlerMapping` with interceptor support, a `RequestMappingHandlerAdapter` with converters and validation, etc.

> This mechanism — *"use the bean if present, else fall back to a default"* — is the ancestor of what Spring Boot later called **auto-configuration**.

A Note on 404s

Since Spring 6.1, when no handler matches a URL, `DispatcherServlet` always raises `NoHandlerFoundException` and allows it to be handled through a `HandlerExceptionResolver` — so a missing mapping surfaces as a proper exception you can handle, rather than a silent container 404.

Putting It All Together — The Minimal App

Controller

```
@Controller
public class HelloController {

    @GetMapping("/hello")
    public String hello(@RequestParam(defaultValue = "World") String name,
                        Model model) {
        model.addAttribute("name", name);
        return "hello";                  // logical view name
    }
}
```

View — `/WEB-INF/views/hello.jsp`

```
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
<html>
<body>
    <h1>Hello, <c:out value="${name}"/>!</h1>
```

```
</body>
</html>
```

Request: `GET /hello?name=Ahmed` → **Response:** `Hello, Ahmed!`

Trace That One Request

`GET /hello?name=Ahmed`

Step	Component	What happens
1	`DispatcherServlet`	receives it (mapped to `/`)
2	`RequestMappingHandlerMapping`	finds `HelloController.hello()` via `@GetMapping("/hello")`
3	—	wraps it in a `HandlerExecutionChain` (no interceptors yet)
4	`RequestMappingHandlerAdapter`	`supports()` → yes, it's an `@RequestMapping` method
5	same adapter	binds `name=Ahmed` → the `String name` parameter; supplies the `Model`
6	`HelloController`	puts `name` in the model, returns `"hello"`
7	`InternalResourceViewResolver`	`"hello"` → `/WEB-INF/views/hello.jsp`
8	`JstlView`	forwards to the JSP, which reads `${name}`
9	`DispatcherServlet`	returns the rendered HTML

Session 1 Recap

- **MVC** is a pattern; **Spring MVC** is Spring's implementation of it, built on servlets
- `DispatcherServlet` is the Front Controller — one entry point, delegates everything
- `ServletContext` is the Servlet API's container (one per WAR); Spring stores its own contexts inside it as attributes
- **Root context** (services) is the **parent** of the **Web context** (controllers); child sees parent, not vice versa
- The root context is **optional** — with no `ContextLoaderListener`, every bean lives in the one web context, which is how Spring Boot works

- Defaults: `/WEB-INF/applicationContext.xml` for the root, `/WEB-INF/<servlet-name>-servlet.xml` for each dispatcher
- **Three ways to register the dispatcher**: `web.xml`, `WebApplicationInitializer`, or the `AbstractAnnotationConfig…` base class — all equivalent at runtime
- `HandlerMapping` = *which* handler (+ interceptors, as a `HandlerExecutionChain`)
- `HandlerAdapter` = *how* to invoke that handler — the **Adapter pattern**, because handlers come in several shapes
- **Controllers**: `@Controller` (modern), `Controller` / `AbstractController` (legacy), `HttpRequestHandler` — three shapes, three adapters
- **Model / ModelMap / ModelAndView** carry data to the view
- `ViewResolver` = logical name → real `View`
- If you define no beans, `DispatcherServlet.properties` supplies **defaults** — but declaring **one** bean of a strategy type switches the defaults **off for that strategy**
- The three seams exist for **loose coupling**; the proof is that the handler style changed three times without `DispatcherServlet` changing